

LabTime — Guion de presentación y defensa

Proyecto Integrador · Arquitectura Empresarial · PUCE · Sábado 18 de julio de 2026

Tiempo total: 7–10 minutos · 8 diapositivas · ambos integrantes deben hablar

0. Antes de empezar

- Repártanse las diapositivas ahora, no en el momento (ver reparto sugerido abajo).
- El que no está hablando debe seguir escuchando — les pueden preguntar sobre la parte del otro.
- Si no saben una respuesta: **no inventen**. Digan “no lo definimos a ese nivel de detalle, pero lo resolveríamos así: ...” y den una idea razonada. Los profesores prefieren eso a una respuesta inventada que se cae con la siguiente pregunta.
- Practiquen en voz alta al menos una vez completa, cronometrada.

Reparto sugerido (2 integrantes: A y B)

Diapositiva	Habla
1. Portada	A (breve, 15 seg)
2. Problema / idea	A
3. Actores y roles	B
4. Diagrama ER	A (la más larga — es el corazón de la entrega)
5. Modelo de autorización	B
6. Matriz de endpoints	A
7. Pantallas de la app	B
8. Cierre	A y B juntos (cada uno dice una parte)

Ajusten según quién domina más cada tema, pero **ambos deben poder responder preguntas de cualquier parte** — así lo dice la rúbrica.

1. Guion por diapositiva

Diapositiva 1 — Portada (15 seg)

*“Buenos días/tardes, somos [nombre A] y [nombre B]. Les presentamos el diseño de **LabTime**, nuestro proyecto integrador para reserva de laboratorios y aulas de la universidad.”*

No se detengan aquí. Es solo la entrada.

Diapositiva 2 — El problema / la idea (40–50 seg)

“El problema que resolvemos: profesores y estudiantes piden laboratorios o aulas por correo o de palabra. No hay un registro único de disponibilidad, se cruzan horarios, y nadie sabe si su solicitud fue aprobada.”

LabTime centraliza esto: el personal de administración académica publica los horarios disponibles por sala, el solicitante — profesor o estudiante — reserva un horario libre, y el personal aprueba o rechaza. Todo queda visible en la app, sin llamadas ni cruces.”

Punto clave a memorizar: el problema es *falta de registro único + sin visibilidad del estado*. Si les preguntan “¿por qué no simplemente un Excel compartido?”, su respuesta es: un Excel no valida permisos, no evita que dos personas reserven lo mismo a la vez, y no separa quién puede aprobar de quién puede solicitar.

Diapositiva 3 — Actores y roles (40-50 seg)

“Definimos dos roles en Cognito. **STAFF**, que es la administración académica: publica salas y horarios, aprueba o rechaza solicitudes, y marca reservas como atendidas. Y **REQUESTER**, que es el profesor o estudiante: busca salas, solicita reservar, cancela solo sus propias reservas, y puede pedir equipo adicional para su reserva.

Sin cuenta, cualquiera puede ver qué salas y horarios existen — eso es público.”

Punto clave: dejen claro que STAFF y REQUESTER **no son tablas**, viven en Cognito como atributo del JWT (`role` claim). Esto es algo que la rúbrica pide explícitamente que quede claro.

Diapositiva 4 — Diagrama ER (90-120 seg, la más importante — 30 de 100 puntos)

Vayan tabla por tabla, señalando la pantalla:

“Tenemos 4 entidades, dentro del rango de 3 a 5 que pide la rúbrica, y ninguna es tabla de usuarios.

`rooms` — la sala o laboratorio: `id`, nombre, tipo (LAB o AULA), capacidad, edificio.

`time_slots` — un horario específico de una sala: tiene `room_id` como FK hacia `rooms`, hora de inicio, hora de fin, y un booleano `available` que indica si sigue libre. Una sala tiene muchos horarios — relación uno a muchos.

`bookings` — la reserva en sí: `slot_id` como FK hacia `time_slots`, el `requester_username` — que viene del JWT, no es FK a ninguna tabla de usuarios porque no existe —, el propósito, el estado (`status`) y cuándo se creó. Un horario puede recibir muchas reservas a lo largo de su historial, aunque solo una esté activa a la vez.

`equipment_requests` — el equipo que se pide para una reserva: `booking_id` como FK hacia `bookings`, nombre del equipo y cantidad. Una reserva puede pedir varios equipos.”

Punto clave sobre `status` : mencionen el flujo `PENDING → APPROVED/REJECTED → ATTENDED` . Si preguntan por qué no es booleano, ya tienen la respuesta (ver sección de preguntas).

Diapositiva 5 — Modelo de autorización (45-60 seg)

“Usamos autorización combinada: **por rol** y **por propiedad**. El rol decide qué tipo de operación puede intentar alguien — por ejemplo, solo STAFF puede publicar salas. La propiedad decide si puede tocar un recurso específico — por ejemplo, un REQUESTER solo puede cancelar su propia reserva, no la de otro.

Sin token en un endpoint privado, devolvemos 401. Con token válido pero rol equivocado, 403. Y si el rol es correcto pero el recurso no le pertenece, también 403, pero ese lo lanza nuestro service, no

Spring Security.”

Este es el punto que más preguntan. Tengan clarísima la diferencia 401 vs 403 (ver sección de preguntas).

Diapositiva 6 — Matriz de endpoints (45-60 seg)

No lean la tabla completa en voz alta — sería aburrido. Señalen 3-4 filas representativas:

“Aquí está la matriz completa. Como ejemplo: `GET /rooms` es público, cualquiera lo consulta. `POST /bookings` requiere ser REQUESTER. Y `DELETE /bookings/{id}` requiere ser REQUESTER y además ser el dueño de esa reserva — ahí es donde entra la validación por propiedad que mencionamos.”

Diapositiva 7 — Pantallas de la app móvil (40-50 seg)

“En la app tendríamos: login contra Cognito, una pantalla para buscar salas y horarios libres, una para reservar, ‘mis reservas’ donde el solicitante ve y cancela las suyas, una para pedir equipo, y del lado de STAFF, gestión de salas/horarios y aprobación de solicitudes. Cada pantalla consume los endpoints que ya mostramos.”

Diapositiva 8 — Cierre (40-50 seg, entre los dos)

A: “Lo más difícil que anticipamos es la concurrencia: si dos personas intentan reservar el mismo horario al mismo tiempo, podríamos terminar con dos reservas para un solo espacio.”

B: “Lo resolvemos marcando `available = false` en la misma transacción en la que se crea la reserva, más una restricción a nivel de base de datos que impide dos reservas activas sobre el mismo horario. Con eso cerramos — gracias, quedamos abiertos a preguntas.”

2. Preguntas que probablemente les hagan (con respuesta lista)

El profesor dijo textualmente que preguntará cosas como “¿por qué esta tabla y no un campo?”, “¿este endpoint por qué es público?”, “¿este 403 de dónde sale?”. Aquí están esas exactas, adaptadas a LabTime, más otras muy probables.

Sobre el diagrama ER

¿Por qué `time_slots` es una tabla y no un campo dentro de `rooms` ? > Porque una sala tiene muchos horarios a lo largo del tiempo, cada uno con su propio estado (disponible u ocupado). Si fuera un campo de `rooms`, solo podríamos guardar un horario por sala — no podríamos representar que el Lab 3 tiene horarios libres el lunes a las 8am y ocupado el martes a las 10am al mismo tiempo.

¿Por qué `equipment_requests` es una tabla separada y no un campo de `bookings` ? > Porque una reserva puede necesitar varios equipos distintos (proyector, cables, laptops), cada uno con su propia cantidad. Es una relación uno a muchos: una reserva, muchas solicitudes de equipo. Meterlo como campo limitaría a un solo equipo por reserva.

¿Por qué `status` es VARCHAR y no un booleano? > Porque una reserva pasa por más de dos estados: `PENDING`, `APPROVED`, `REJECTED`, `ATTENDED` (y posiblemente `CANCELLED`). Un booleano solo distingue dos valores, y aquí necesitamos representar todo el ciclo de vida de la aprobación.

¿Por qué no crearon una tabla `buildings` para el edificio? > Porque en este alcance el edificio es solo un dato descriptivo de la sala (un texto), no tiene atributos ni comportamiento propio que justifique su propia tabla. Además nos habría sacado del rango de 3 a 5 entidades. Si el proyecto creciera y necesitaríamos administrar edificios de forma independiente, ahí sí lo normalizaríamos.

¿Por qué `requester_username` es `VARCHAR` y no una `FK`? > Porque no existe tabla de usuarios en nuestro backend — los usuarios viven en Cognito, no en nuestra base de datos. Guardamos el username tal como viene en el claim del JWT, y lo usamos para comparar identidad (por ejemplo, al validar que alguien solo cancele su propia reserva), no como una relación de base de datos.

¿Qué diferencia hay entre `available` en `time_slots` y `status` en `bookings`? > `available` es un flag del horario: indica si ese horario sigue libre para reservarse. `status` es el estado de la reserva en sí, y sigue su propio flujo (pendiente, aprobada, atendida) independientemente de que el slot ya esté marcado como no disponible.

Sobre los endpoints

¿Por qué `GET /rooms` y `GET /rooms/{id}/slots/available` son públicos? > Porque cualquier persona, incluso sin cuenta, debería poder ver qué salas y horarios existen antes de decidir si se registra o solicita una reserva. Es información de solo lectura, sin datos sensibles — como el catálogo de un sistema de reservas.

¿Por qué `POST /bookings` no es público? > Porque crear una reserva asocia el recurso a un usuario específico (`requester_username` viene del JWT) — sin autenticación no sabríamos a quién pertenece la reserva, y cualquiera podría spamear el sistema con reservas falsas.

¿Por qué `DELETE /bookings/{id}` necesita rol y propiedad, y no solo rol? > Porque si solo validáramos el rol `REQUESTER`, cualquier estudiante podría cancelar la reserva de otro estudiante con solo tener una cuenta. La validación de propiedad asegura que *solo* el dueño de esa reserva específica pueda cancelarla.

Sobre autenticación y autorización

¿De dónde sale exactamente el 403 en `DELETE /bookings/{id}` cuando no es tu reserva? > No lo lanza Spring Security — lo lanza nuestro **service**. Spring Security (con el token de Cognito) ya validó que el usuario tiene rol `REQUESTER`, así que deja pasar la petición al controller. Dentro del service, comparamos `booking.requesterUsername` contra el username que sacamos del JWT (vía `SecurityContext`). Si no coinciden, lanzamos una excepción personalizada que el `GlobalExceptionHandler` traduce a HTTP 403.

¿Cuál es la diferencia entre 401 y 403 en su sistema? > 401 significa que Spring Security no pudo identificar al usuario — no hay token, expiró, o la firma no es válida. Es “no sé quién eres”. 403 significa que sí sabemos quién es (el JWT es válido), pero no tiene permiso: o su rol no alcanza para esa operación, o el recurso específico no le pertenece.

¿Por qué usan Cognito en vez de su propia tabla de usuarios con contraseñas? > Porque implementar autenticación segura (hash de contraseñas, recuperación, expiración de sesión, MFA) es un problema ya resuelto y delicado de seguridad. Cognito nos da JWT firmados y manejo de usuarios fuera de nuestro backend; nosotros solo confiamos en ese token y leemos sus claims (username, rol) para tomar decisiones de autorización.

¿Qué pasa si el JWT es válido pero no tiene el claim de rol? > (Si no lo definieron formalmente, respondan con criterio:) Lo tratamos como acceso denegado por defecto — preferimos fallar cerrado (denegar) que fallar abierto (permitir) cuando falta información de autorización.

Sobre el diseño en general

¿Qué pasa si dos personas reservan el mismo horario al mismo tiempo? > Es el reto que mencionamos en el cierre. Lo resolvemos con una transacción: al crear la reserva, marcamos `available =`

`false` en el mismo `time_slot` dentro de la misma transacción de base de datos, y agregamos una restricción que impide que haya dos reservas activas sobre el mismo `slot_id`. Así, aunque lleguen casi al mismo tiempo, la base de datos solo deja pasar una.

¿Qué capas va a tener el backend? > Las mismas que usamos en los laboratorios del curso: `controllers`, `services`, `repositories`, `entities`, `dto`, `mappers`, `exceptions`, `config` — Kotlin con Spring Boot 4.

¿Cómo van a probar que la autorización funciona? > Con Postman, probando cada fila de la matriz de endpoints: sin token (esperando 401), con token pero rol equivocado (esperando 403), con token y rol correcto pero sin ser el dueño (esperando 403), y con todo correcto (esperando 200/201).

3. Últimos consejos

- **Tiempo:** 8 diapositivas en 7-10 min son ~50-70 seg por diapositiva en promedio. El diagrama ER puede llevarse más (es el 30% de la nota), compensen siendo más breves en portada y pantallas.
- **No se disculpen** por no tener código todavía — el enunciado es explícito en que hoy solo se entrega el diseño.
- **Señalen la pantalla** al hablar del diagrama ER y la matriz — no lean de memoria mirando al profesor, apunten a la tabla o fila específica de la que hablan.
- Si les preguntan algo que **no está en su diseño actual**, está bien decir “no lo cubrimos a ese detalle, pero lo resolveríamos con X” — mostrar criterio importa más que tener todas las respuestas ya escritas.
- Revisen el checklist del enunciado antes de dormir: **subir la presentación en PDF al aula virtual es requisito para que evalúen**, no solo presentarla en clase.